# MID-TERM STUDY GUIDE

Neural Networks, Training, TensorFlow & Deep Learning Basics

---

**Topics covered**

1. Neural Network Basics
2. Training Concepts
3. TensorFlow
4. General ML and Deep Learning

*Every point is explained separately, with diagrams, examples and exam tips.*


AAKASHRAJAWAT75@GMAIL.COM

# 1. Neural Network Basics

A neural network is a computing system loosely inspired by the brain. It is built from many small units (neurons) arranged in layers. Each neuron does a very simple calculation, but together they can learn complex patterns such as recognising faces or translating language.

## 1.1 Artificial Neuron Architecture

The artificial neuron is the basic building block. It receives numbers, combines them, and produces one number as output.

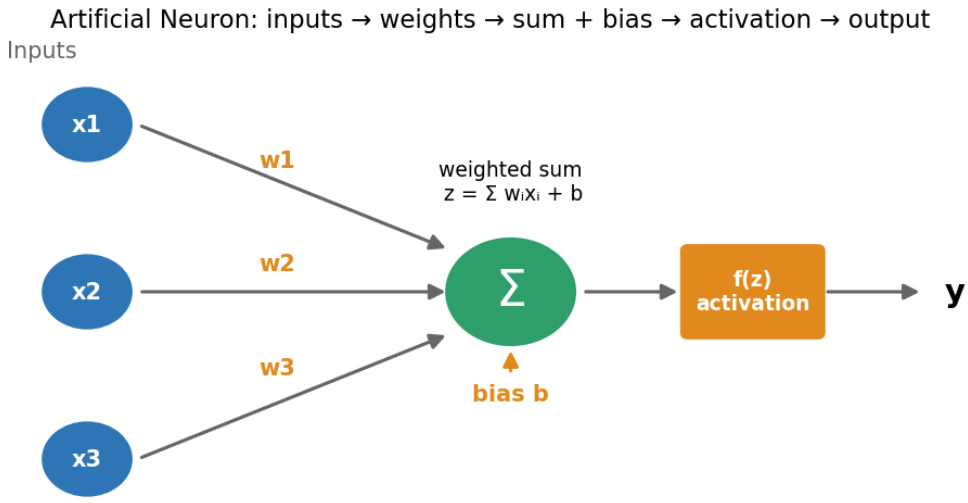



*Figure 1: Structure of an artificial neuron*

### Parts of a neuron

- **Inputs ($x_1, x_2, \ldots x_n$):** the data fed into the neuron (for example pixel values or features).
- **Weights ($w_1, w_2, \ldots w_n$):** numbers that show how important each input is. A large weight means the input has strong influence. Weights are what the network *learns*.
- **Bias (b):** an extra adjustable number added to the sum. It lets the neuron shift its output up or down, so it can fire even when all inputs are zero.
- **Summation (Σ):** computes the weighted sum $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$.
- **Activation function f(z):** transforms z into the final output y = f(z). It adds non-linearity (examples: sigmoid, tanh, ReLU).
- **Output (y):** the result passed to the next layer or given as the final prediction.

### Worked example

Inputs x = (2, 3), weights w = (0.5, −1), bias b = 1. $z = (0.5\times2) + (-1\times3) + 1 = 1 - 3 + 1 = -1$. With ReLU, $y = \max(0, -1) = 0$.

### Common activation functions

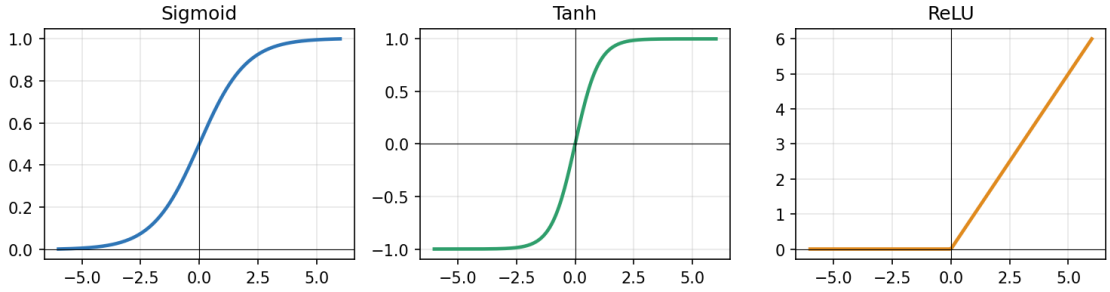



*Figure 2: Sigmoid, Tanh and ReLU*

| Function | Formula | Output range | Typical use |
|---|---|---|---|
| Sigmoid | $1 / (1 + e^{-z})$ | 0 to 1 | Binary classification output |
| Tanh | $(e^{z} - e^{-z})/(e^{z} + e^{-z})$ | −1 to 1 | Hidden layers (older nets) |
| ReLU | max(0, z) | 0 to ∞ | Default for hidden layers |
| SoftMax | $e^{zi} / \Sigma\, e^{zj}$ | 0 to 1, sums to 1 | Multi-class output layer |

## 1.2 Hidden Layer, and Does the Input Layer Use an Activation Function?

### What is a hidden layer?

A hidden layer is any layer **between the input layer and the output layer**. It is called "hidden" because we do not directly see its values: we give inputs and see outputs, but the middle computation is internal. Hidden layers extract features. In an image network, early hidden layers detect edges, later ones detect shapes, and the last ones detect whole objects.

- Each hidden neuron applies weights, bias and an activation function.
- More hidden layers and neurons = more ability to learn complex patterns (but also more risk of overfitting and more computation).

### Does the input layer use an activation function?

**Answer: No.** The input layer does not use an activation function. It only holds/passes the raw input data to the first hidden layer. It has no weights, no bias and performs no computation.

- Input neurons are just placeholders for feature values ($x_1$, $x_2$, …).
- Activation functions are applied from the **first hidden layer onward** (and in the output layer, depending on the task).
- Because of this, the "number of layers" of a network is usually counted **without** the input layer.

## 1.3 Shallow vs. Deep Feedforward Networks

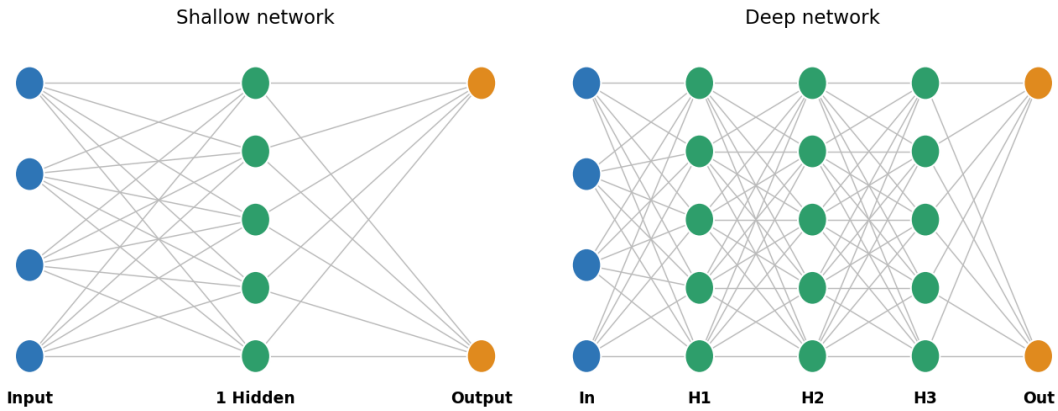



*Figure 3: Shallow network (one hidden layer) vs deep network (many hidden layers)*

| Feature | Shallow network | Deep network |
|---|---|---|
| Hidden layers | One (sometimes two) | Three or more (can be hundreds) |
| Learning ability | Handles simple patterns | Learns complex, hierarchical features |
| Feature engineering | Often needs manual features | Learns features automatically |
| Data needed | Small to moderate | Large amounts |
| Training cost | Low, fast | High, needs GPUs |
| Overfitting risk | Lower | Higher without regularisation |
| Example use | Simple regression, small tabular data | Image, speech, language tasks |

**Key idea:** depth lets the network build complex features out of simpler ones, layer by layer. A shallow network would need an enormous number of neurons to represent the same function.

## 1.4 All Layers in a Feedforward Network

In a feedforward neural network (FNN), information moves in **one direction only**: input → hidden layer(s) → output. There are no loops or cycles.

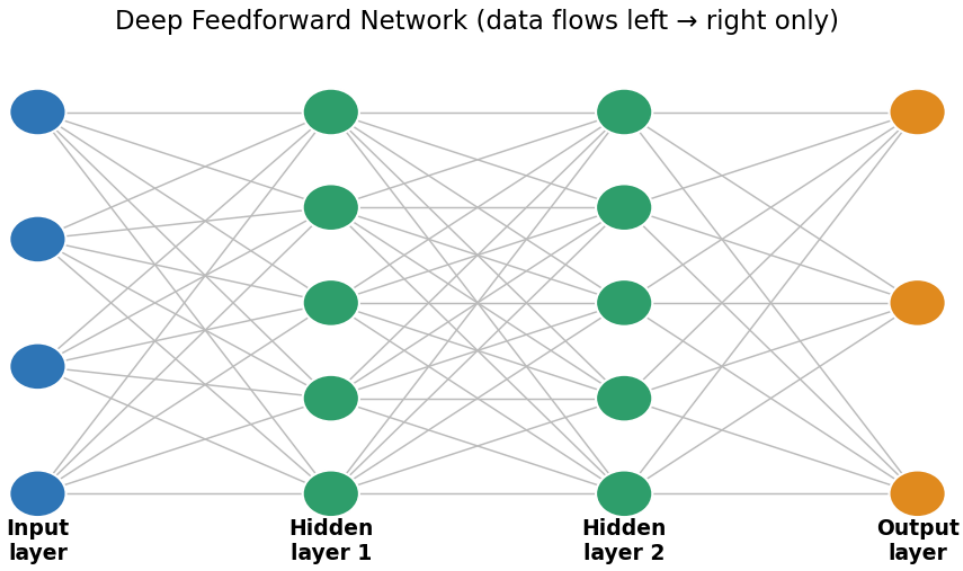



*Figure 4: Layers of a feedforward network*

1. **Input layer:** receives raw data. One neuron per feature (e.g. 784 neurons for a 28×28 image). No computation, no activation.
2. **Hidden layer(s):** do the learning. Each neuron computes weighted sum + bias, then activation. Layers are usually *fully connected (dense)*.
3. **Output layer:** gives the final prediction. Its size and activation depend on the task:
   - Regression → 1 neuron, linear (no) activation.
   - Binary classification → 1 neuron, sigmoid.
   - Multi-class classification → one neuron per class, SoftMax.

**Forward pass:** data flows through all layers to produce a prediction. **Backward pass (backpropagation):** the error is sent back to update weights during training.

## 1.5 Limitations of Linear Neurons

A linear neuron uses f(z) = z (or any straight-line function). It only computes a weighted sum, so its output is a straight-line function of its inputs.

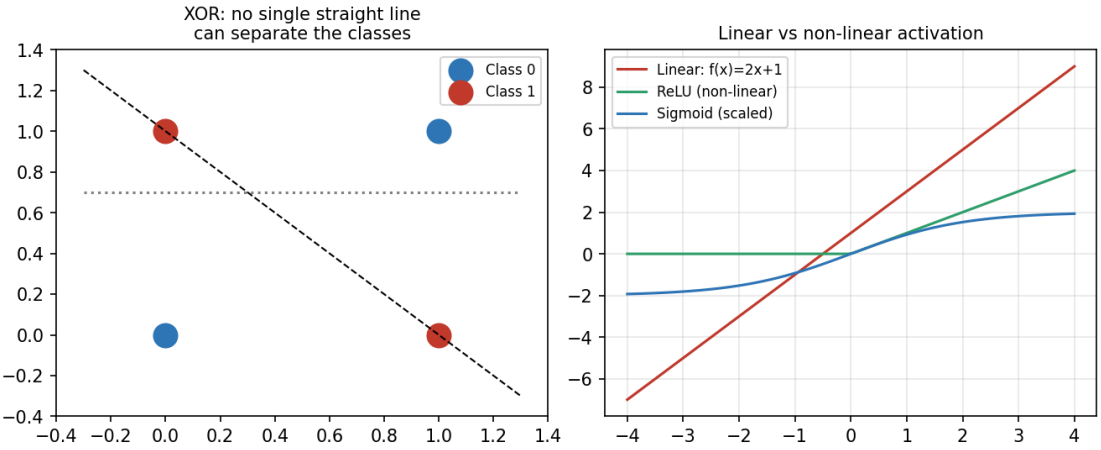



*Figure 5: XOR cannot be separated by a line; non-linear activations bend the output*

### Main limitations

- **Only linear decision boundaries:** it can separate data only with a straight line/plane. Problems like XOR cannot be solved.
- **Stacking does not help:** two linear layers collapse into one. If layer 1 gives $W_1x$ and layer 2 gives $W_2(W_1x)$, the result is $(W_2W_1)x$, which is just another linear function. A 100-layer linear network is no more powerful than a single neuron.
- **Cannot model complex real-world data:** images, speech and text have non-linear relationships.
- **Unbounded output:** a linear output can be any value, unsuitable for probabilities.
- **Constant gradient:** the derivative is the same everywhere, which gives no useful information to learn non-linear structure.

**Solution:** Use non-linear activation functions (ReLU, sigmoid, tanh) in hidden layers. This gives the network the power to approximate almost any function.

# 2. Training Concepts

Training means adjusting the weights and biases so the network's predictions become as close as possible to the true answers. The loop is: **predict → measure error (loss) → compute gradients → update weights → repeat.**

## 2.1 Loss Function

A loss function (also called cost or error function) is a number that measures **how wrong** the model's prediction is compared to the true value. Low loss = good predictions. Training tries to minimise it.

| Loss function | Formula | Used for |
|---|---|---|
| Mean Squared Error (MSE) | $(1/n)\ \Sigma\ (y - \hat{y})^2$ | Regression |
| Mean Absolute Error (MAE) | $(1/n)\ \Sigma\ \lvert y - \hat{y} \rvert$ | Regression (robust to outliers) |
| Binary cross-entropy | $-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$ | Two-class classification |
| Categorical cross-entropy | $-\Sigma\ y_i \log \hat{y}_i$ | Multi-class classification |

**Example:** true value 10, prediction 7 → squared error = $(10 - 7)^2 = 9$. If prediction is 9.5 → error = 0.25. The second model is much better.

## 2.2 Cross Entropy

Cross-entropy measures the difference between two probability distributions: the **true labels** and the **predicted probabilities**. It is the standard loss for classification.

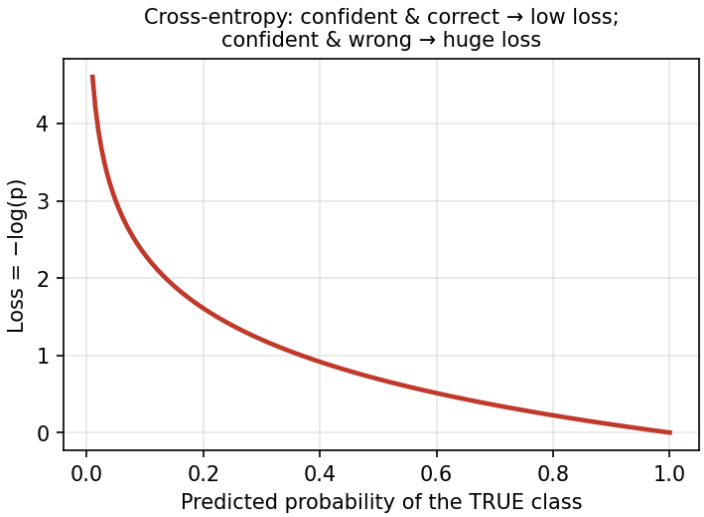



*Figure 6: Cross-entropy loss as a function of predicted probability*

- For one example with true class k, loss = **$-\log(p_k)$**, where $p_k$ is the probability the model gave to the correct class.

- If the model gives the correct class probability 0.9 → loss = −log(0.9) ≈ 0.105 (small).
- If it gives 0.1 → loss = −log(0.1) ≈ 2.30 (large). If it gives 0.01 → loss ≈ 4.6.
- It **punishes confident wrong answers heavily**, pushing the model to be both correct and well calibrated.
- It works well with SoftMax/sigmoid, and gives larger gradients than MSE when the model is badly wrong, so learning is faster.

**Full formula (multi-class):** $L = -\Sigma_i y_i \cdot \log(\hat{y}_i)$, where y is the one-hot true label and ŷ is the predicted probability vector.

## 2.3 SoftMax Classifier

SoftMax converts a vector of raw scores (called **logits**) into **probabilities** that are all between 0 and 1 and **add up to 1**. It is used in the output layer for multi-class problems.

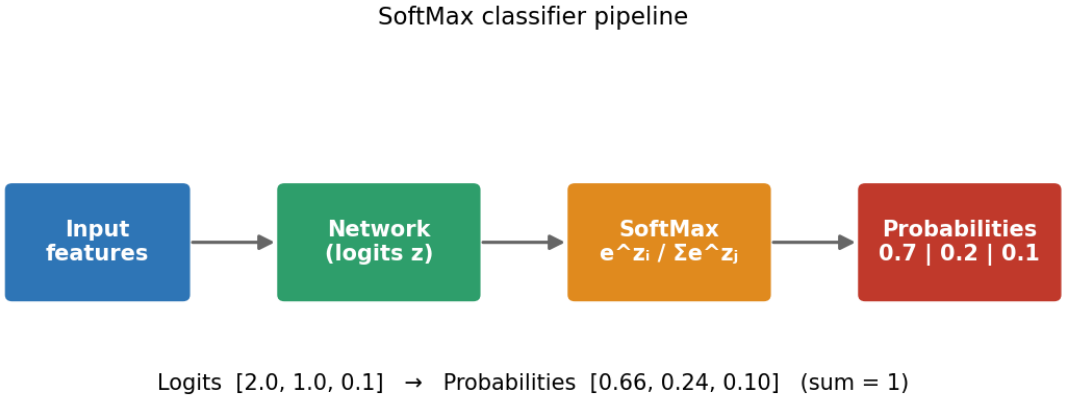



*Figure 7: SoftMax turns logits into probabilities*

**Formula:** $\text{softmax}(z_i) = e^{z_i} / \Sigma_j e^{z_j}$

### Worked example

Logits z = [2.0, 1.0, 0.1]. $e^2 = 7.39$, $e^1 = 2.72$, $e^{0.1} = 1.11$. Sum = 11.22. Probabilities = [0.66, 0.24, 0.10]. The class with the highest probability (class 1) is the prediction.

- The exponential makes all values positive and exaggerates differences between scores.
- A SoftMax classifier = a network (or linear layer) + SoftMax + cross-entropy loss.
- Sigmoid is for 2 classes (or multi-label); SoftMax is for many mutually exclusive classes.

## 2.4 Gradient Descent

Gradient descent is the optimisation algorithm that **minimises the loss**. Imagine standing on a foggy hill and wanting to reach the valley: you feel the slope under your feet and take a step downhill. Repeat until you reach the bottom.

**Update rule:** $w_{new} = w_{old} - \eta \times \partial L/\partial w$

- **∂L/∂w (gradient):** the slope of the loss with respect to the weight. It points toward increasing loss, so we move in the opposite direction (minus sign).
- **η (eta):** the learning rate, which controls the step size.

### Steps

1. Initialise weights randomly.
2. Forward pass: compute predictions.
3. Compute loss.
4. Compute gradients using backpropagation.
5. Update weights with the rule above.
6. Repeat until the loss stops decreasing.

### Variants

| Variant | Data used per update | Characteristic |
|---|---|---|
| Batch gradient descent | Entire dataset | Stable but slow; heavy memory use |
| Stochastic (SGD) | 1 example | Fast, noisy, can escape local minima |
| Mini-batch | Small group (e.g. 32–256) | Best balance; most commonly used |

## 2.5 Learning Rate

The learning rate (η) is a **hyperparameter** that decides how big each weight-update step is. It is one of the most important settings in training.

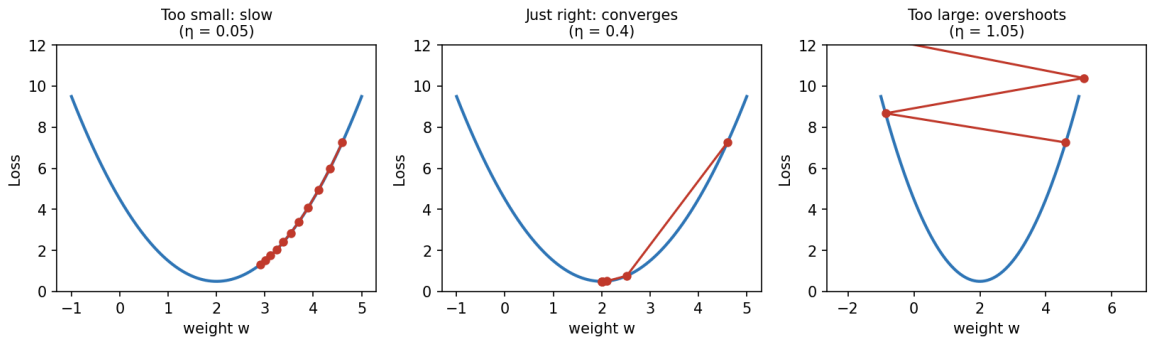



*Figure 8: Effect of learning rate on gradient descent*

- **Too small:** training is extremely slow and may get stuck in a poor spot.
- **Too large:** steps overshoot the minimum; the loss bounces around or even grows (diverges).
- **Just right:** loss decreases smoothly and converges.
- Typical starting values: 0.1, 0.01, 0.001. Learning-rate schedules and adaptive optimisers (Adam, RMSprop) adjust it automatically.

## 2.6 Maxima and Minima

These are special points on the loss curve where the slope (gradient) is **zero**.

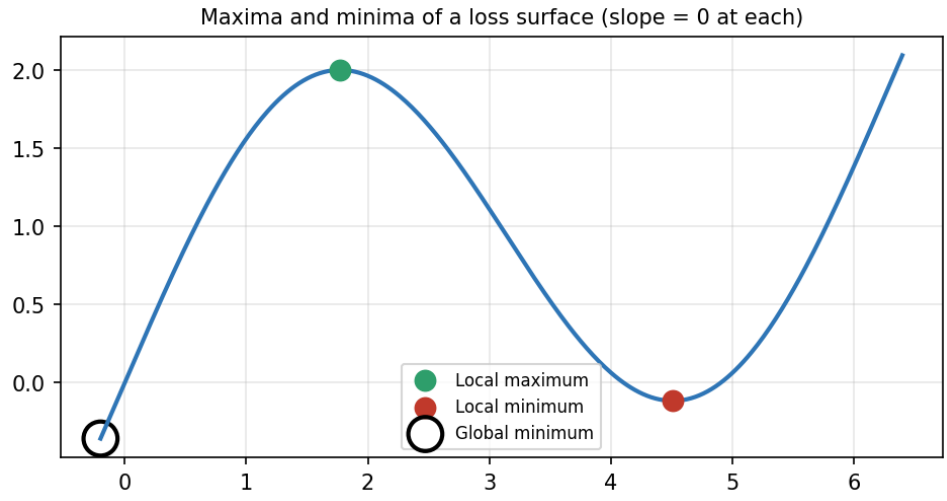



*Figure 9: Local and global maxima / minima*

- **Minimum:** a low point. A **local minimum** is lower than its neighbours; the **global minimum** is the lowest point of the whole function. Training aims for the global (or a good local) minimum.
- **Maximum:** a high point (local or global). We want to avoid being here, as loss is high.
- **Saddle point:** gradient is zero but it is a minimum in one direction and a maximum in another. These are common in high-dimensional loss surfaces and can slow training.
- **Test:** at a critical point, if the second derivative > 0 → minimum; if < 0 → maximum.

**Why it matters:** Gradient descent can get trapped in a local minimum or slow down on a flat area. SGD noise, momentum and good initialisation help escape them. In deep networks, most local minima are about as good as the global one.

# 3. TensorFlow

## 3.1 What is TensorFlow?

TensorFlow is an **open-source machine learning framework developed by Google Brain** (released 2015). It is used to build, train and deploy ML and deep-learning models.

- Does fast numerical computation on **tensors** using CPUs, GPUs and TPUs.
- Provides **automatic differentiation**, so gradients for backpropagation are computed for you.
- Works from research to production: desktop, servers, mobile (TensorFlow Lite), browser (TensorFlow.js).
- Supports Python (main), C++, JavaScript, and more. Includes TensorBoard for visualising training.

The name comes from its core idea: **tensors flowing** through a graph of operations.

## 3.2 Keras vs. TensorFlow

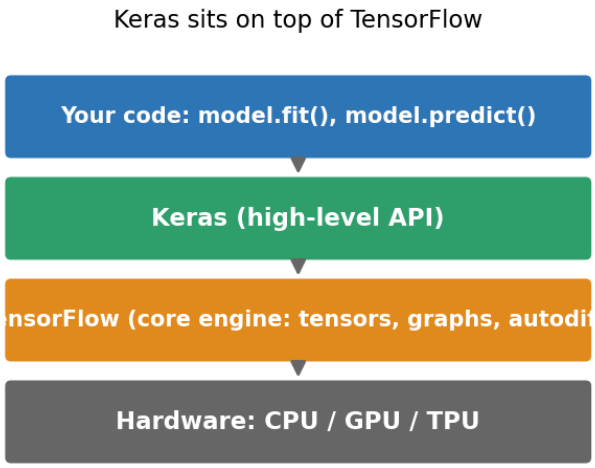



*Figure 10: Keras is a high-level API on top of TensorFlow*

| Aspect | TensorFlow | Keras |
|---|---|---|
| What it is | Complete low-level + high-level ML platform | High-level neural network API |
| Level | Lower level, more control | Simple, user-friendly |
| Purpose | Tensor math, graphs, autodiff, deployment | Quickly build and train models |
| Code length | Longer for custom work | Few lines (Sequential, Dense, fit) |
| Relationship | The engine (backend) | Runs on top of TensorFlow (tf.keras) |
| Best for | Research, custom ops, production pipelines | Beginners, rapid prototyping |

Since TensorFlow 2.0, Keras is the **official high-level API** of TensorFlow, available as `tf.keras`.

### Simple Keras example

```
import tensorflow as tf
model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation='relu', input_shape=(4,)),
    tf.keras.layers.Dense(3, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy')
model.fit(X, y, epochs=10)
```

## 3.3 Tensors

A tensor is a **multi-dimensional array** of numbers with a uniform data type. It is the basic data structure in TensorFlow: all inputs, outputs, weights and intermediate results are tensors.

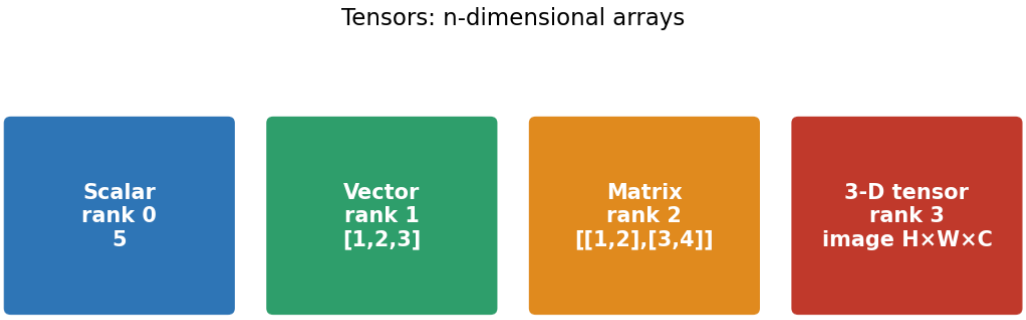



*Figure 11: Tensor ranks*

| Name | Rank (dims) | Example | Shape |
|---|---|---|---|
| Scalar | 0 | 5 | () |
| Vector | 1 | [1, 2, 3] | (3,) |
| Matrix | 2 | [[1,2],[3,4]] | (2, 2) |
| 3-D tensor | 3 | Colour image | (height, width, channels) |
| 4-D tensor | 4 | Batch of images | (batch, h, w, channels) |

- **Rank:** number of dimensions. **Shape:** size along each dimension. **dtype:** data type (float32, int32, …).
- `tf.constant()` creates an immutable tensor; `tf.Variable()` creates a tensor that can change (used for weights).
- Tensors can live on CPU or GPU memory for faster math.

## 3.4 Computational Graph

A computational graph is a **directed graph that describes a calculation**. TensorFlow turns your code into such a graph before (or while) running it.

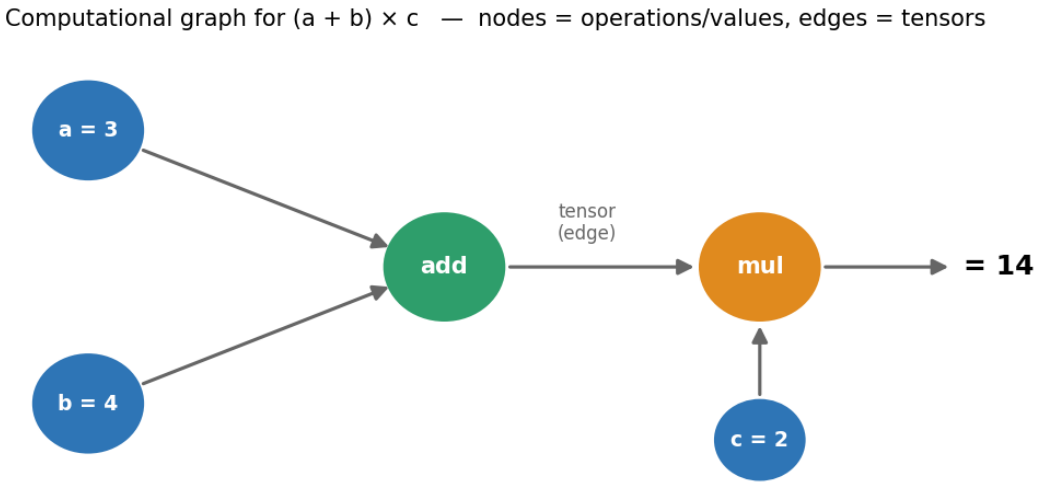



*Figure 12: Computational graph of (a + b) × c*

### Nodes and edges

- **Nodes:** operations (add, multiply, matmul) or values (constants, variables).
- **Edges:** the **tensors** that flow between nodes, carrying output of one operation into the next. Their direction shows data dependency.

### Why is it used?

- **Automatic differentiation:** the graph records every operation so gradients can be computed by walking it backwards (backpropagation).
- **Optimisation:** TensorFlow can simplify, fuse and reorder operations for speed.
- **Parallelism:** independent nodes can run at the same time on multiple CPUs, GPUs or TPUs.
- **Portability:** a graph can be saved and deployed to mobile, web or servers without Python code.
- **Distribution:** different parts of the graph can run on different machines.

## 3.5 TensorFlow 1.0 vs. 2.0

TensorFlow 2.0 was a major redesign focused on simplicity and ease of use.

| Feature | TensorFlow 1.x | TensorFlow 2.x |
|---|---|---|
| Execution mode | Static graph; build graph first, then run in a Session | Eager execution by default; runs immediately like normal Python |
| Placeholders | tf.placeholder() to feed data at run time | Removed; pass data directly as arguments to functions/models |
| Variable initialisation | tf.global_variables_initializer() required before running | Not needed; variables are initialised on creation |

| Feature | TensorFlow 1.x | TensorFlow 2.x |
|---|---|---|
| Printing values | tf.Print() op inside the graph, or evaluate with sess.run() | tf.print() works immediately; normal print() also shows values in eager mode |
| Sessions | tf.Session() and sess.run() required | No sessions needed |
| High-level API | Mixed (tf.layers, contrib, separate Keras) | tf.keras is the single official API |
| Debugging | Hard (graph is hidden) | Easy (use Python tools, step through code) |
| Graph still available? | Always | Yes, via @tf.function for speed |

### Details of the four exam points

- **Placeholders (1.x):** empty "slots" in the graph that you fill with data at run time using feed_dict. Replaced in 2.0 by ordinary function arguments.
- **tf.global_variables_initializer() (1.x):** an operation that must be run in a session to give all tf.Variable objects their initial values. It is unnecessary in 2.0.
- **Eager execution (2.0):** operations are evaluated instantly and return concrete values, with no need to build a graph and run a session first. This makes TensorFlow feel like NumPy.
- **tf.print() (2.0):** prints tensor values and works in both eager mode and inside @tf.function graphs. Unlike Python print(), it still prints actual values when the code is compiled into a graph.

### Code comparison

**TensorFlow 1.x**

```
a = tf.placeholder(tf.float32)
b = tf.placeholder(tf.float32)
c = a + b
with tf.Session() as sess:
    print(sess.run(c, feed_dict={a: 3.0, b: 4.0}))   # 7.0
```

**TensorFlow 2.x**

```
a = tf.constant(3.0)
b = tf.constant(4.0)
c = a + b
tf.print(c)      # 7
```

# 4. General ML and Deep Learning

## 4.1 Qualities of a Good Machine Learning Method

A good ML method is not only accurate on training data. It should perform well in the real world. Key qualities:

- **Accuracy / predictive performance:** makes correct predictions on new, unseen data.
- **Generalisation:** avoids overfitting (memorising training data) and underfitting (being too simple). Good performance on the test set, not just the training set.
- **Robustness:** still works with noisy data, missing values and outliers.
- **Scalability / efficiency:** training and prediction time and memory grow reasonably as data grows.
- **Interpretability:** humans can understand why it made a decision (important in medicine, finance, law).
- **Simplicity:** the simplest model that does the job is preferred (Occam's razor); fewer parameters mean less risk of overfitting.
- **Low data requirement:** performs well even without huge labelled datasets.
- **Stability / reproducibility:** small changes in data do not wildly change the model; results can be repeated.
- **Adaptability:** can be updated when new data arrives.
- **Fairness:** not biased against any group.

## 4.2 Hardware Requirements: ML vs. Deep Learning

| Aspect | Traditional machine learning | Deep learning |
|---|---|---|
| Processor | Standard CPU is enough | GPU / TPU strongly needed for speed |
| Data volume | Works on small and medium datasets | Needs very large datasets |
| RAM / memory | Modest (a few GB) | High (tens to hundreds of GB; GPU VRAM matters) |
| Training time | Minutes to hours | Hours to weeks |
| Computation | Simpler algorithms (SVM, trees, regression) | Millions–billions of parameters; heavy matrix maths |
| Storage | Small | Large (datasets, model checkpoints) |
| Cost | Low; a laptop works | High; GPU servers or cloud |
| Feature engineering | Manual, done by humans | Automatic, learned by network (hence more compute) |

**Why GPUs?** Deep learning is mostly huge matrix multiplications. GPUs have thousands of small cores that do these in parallel, making training many times faster than a CPU. TPUs are Google's custom chips designed specifically for tensor operations.

## 4.3 Stages of the Data Science Lifecycle

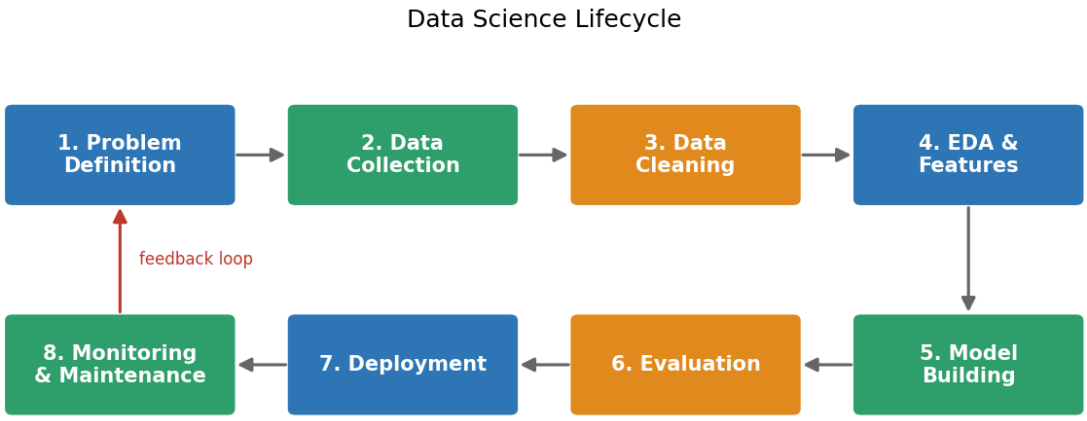



*Figure 13: Data science lifecycle*

1. **Problem definition (business understanding):** decide the goal and what success looks like. Example: "Predict which customers will leave."
2. **Data collection:** gather data from databases, APIs, sensors, web scraping, surveys, logs.
3. **Data cleaning / preparation:** handle missing values, duplicates, errors and outliers; fix formats; this often takes most of the project time.
4. **Exploratory data analysis (EDA) & feature engineering:** visualise and summarise data, find patterns, create and select useful features, scale/encode variables.
5. **Model building:** choose algorithms, split data into train/validation/test, train the model and tune hyperparameters.
6. **Model evaluation:** measure performance with suitable metrics (accuracy, precision, recall, F1, RMSE) on unseen test data and check it meets the goal.
7. **Deployment:** put the model into production (web service, app, API) so users or systems can use it.
8. **Monitoring & maintenance:** track performance over time, detect data drift, retrain with fresh data and improve. The cycle then repeats.

**Note:** The lifecycle is **iterative**. Results at later stages often send you back to earlier ones (for example, poor evaluation → collect more data or change features). Different textbooks may merge or rename stages, but the flow is the same.

# Quick Revision Cheat Sheet

| Topic | One-line memory |
|---|---|
| Artificial neuron | output = activation( Σ weight × input + bias ) |
| Hidden layer | Any layer between input and output; extracts features |
| Input layer activation | NO activation; only passes raw data |
| Shallow vs deep | Shallow = 1 hidden layer; deep = many hidden layers |
| Feedforward layers | Input → hidden(s) → output; one-way flow |
| Linear neuron limit | Only straight boundaries; stacked linear layers collapse to one |
| Loss function | Number showing how wrong the prediction is |
| Cross entropy | −log(probability of true class); classification loss |
| SoftMax | Scores → probabilities that sum to 1 |
| Gradient descent | w = w − η × gradient |
| Learning rate | Step size: too small = slow, too large = diverge |
| Maxima / minima | Points where gradient = 0; we seek the minimum of loss |
| TensorFlow | Google's open-source ML framework built on tensors and graphs |
| Keras vs TF | Keras = easy high-level API on top of TensorFlow |
| Tensor | n-dimensional array (scalar, vector, matrix, …) |
| Computational graph | Nodes = operations, edges = tensors; enables autodiff and parallelism |
| TF 1 vs TF 2 | Placeholders, sessions, initializer → eager execution, tf.print() |
| Good ML method | Accurate, generalises, robust, scalable, interpretable, simple |
| ML vs DL hardware | ML: CPU is fine. DL: GPU/TPU, lots of RAM and data |
| Lifecycle | Problem → Data → Clean → EDA → Model → Evaluate → Deploy → Monitor |